

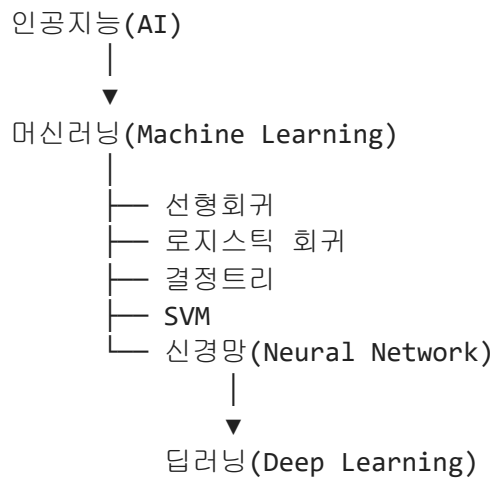
00. AI, 머신러닝, 딥러닝과 신경망 관계

장 큰 개념은 인공지능(AI) 이고, 그 안에 머신러닝, 그 안에 딥러닝, 그리고 딥러닝을 구성하는 핵심 기술이 신경망입니다.

신경망(Neural Network) 은 사람의 뇌를 모방하여 만든 머신러닝 모델이고, **딥러닝(Deep Learning)**은 이러한 신경망을 여러 층으로 깊게 쌓아 복잡한 문제를 해결하는 기술입니다.

1. 머신러닝과 딥러닝의 차이

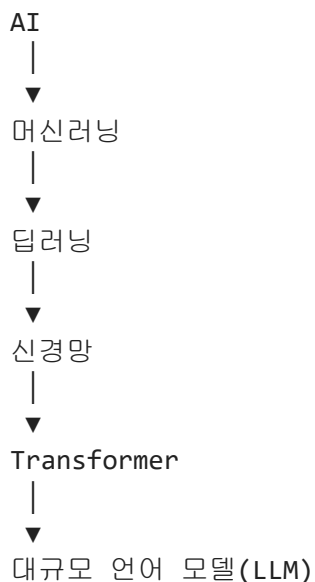
먼저 큰 그림부터 보면 다음과 같습니다.



- **AI** : 사람처럼 학습하고 판단하는 컴퓨터 기술 전체를 의미
- **머신러닝** : AI를 만드는 방법 중 하나
- **신경망** : 뉴런을 여러 개 연결한 모델
- **딥러닝** : 신경망의 층(Layer)이 매우 많은 모델

2. ChatGPT는 어디에 속할까?

ChatGPT는 신경망을 기반으로 하는 딥러닝 모델입니다.



↓
ChatGPT

In []:

01. 신경망이란?

신경망(Neural Network)은 사람의 뇌가 정보를 처리하는 방식을 본떠 만든 인공지능 학습 모델입니다.

사람은 경험을 통해 배우듯이, 신경망도 많은 데이터를 보면서 규칙을 스스로 학습합니다.

예를 들어,

- 고양이 사진을 수천 장 보여주면 → 고양이를 구별하는 방법을 배우고,
- 사람 목소리를 많이 들려주면 → 음성을 인식하는 방법을 배우며,
- 문장을 많이 읽으면 → 번역하거나 글을 작성하는 방법을 배웁니다.

사람의 뇌와 신경망

사람의 뇌

뉴런 — 뉴런 — 뉴런
↓ ↓ ↓
정보 전달 → 학습 → 판단

↓

인공 신경망

입력층 — 은닉층 — 출력층
↓ ↓ ↓
데이터 → 계산과 학습 → 결과

사람의 뇌에는 약 860억 개의 뉴런이 연결되어 있고, 인공 신경망은 이러한 구조를 수학적으로 단순화하여 컴퓨터에서 구현한 모델입니다.

신경망은 어떻게 학습할까?

신경망은 다음 과정을 반복하면서 학습합니다.

데이터 입력
↓
예측
↓
정답과 비교
↓

오차 계산



가중치 수정



다시 예측

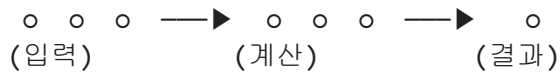
이 과정을 수천 번, 수만 번 반복하면서 점점 더 정확한 결과를 만들어 냅니다.

신경망의 구조

입력층

은닉층

출력층



- **입력층(Input Layer)** : 데이터를 입력받습니다.
- **은닉층(Hidden Layer)** : 데이터를 분석하고 특징을 학습합니다.
- **출력층(Output Layer)** : 최종 결과를 출력합니다.

신경망이 사용되는 곳

신경망은 우리 주변의 다양한 AI 기술에 사용됩니다.

- 📱 얼굴 인식(스마트폰 잠금 해제)
- 🎤 음성 인식(음성 비서)
- 🚗 자율주행 자동차
- 🌐 번역 서비스
- 💬 ChatGPT와 같은 생성형 AI
- 📷 사진 속 사물 인식

신경망을 설명하기 좋은 비유

신경망을 시험공부에 비유하면 이해하기 쉽습니다.

문제를 푼다.



틀린 문제를 확인한다.



왜 틀렸는지 공부한다.



다시 문제를 푼다.



점점 성적이 오른다.

신경망도 똑같습니다.

데이터를 입력한다.



예측한다.



틀린 정도(오차)를 계산한다.



가중치를 수정한다.



점점 더 정확해진다.

한 문장으로 정리

신경망은 사람의 뇌를 본떠 만든 인공지능 모델로, 많은 데이터를 반복해서 학습하며 스스로 규칙을 찾아 예측과 판단을 수행하는 기술입니다.

이 정의를 바탕으로 다음 단계에서는 "인공 뉴런(Artificial Neuron)"을 설명하면 학생들이 신경망의 구조와 학습 원리를 자연스럽게 이해할 수 있습니다.

In []:

02. 인공 뉴런의 구조

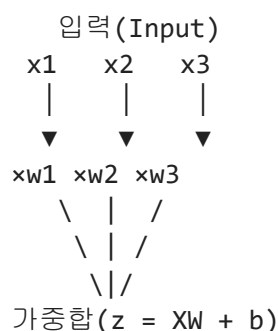
중·고등학생을 대상으로 한다면 복잡한 수식보다 "입력 → 계산 → 판단 → 출력"의 흐름으로 설명하는 것이 가장 이해하기 쉽습니다.

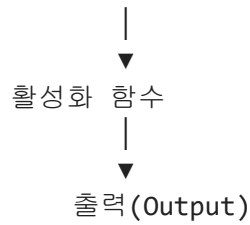
인공 뉴런(Artificial Neuron)의 구조

인공 뉴런은 신경망을 구성하는 가장 작은 계산 단위입니다.

사람의 뇌에서 뉴런이 정보를 받아 처리하듯이, 인공 뉴런도 입력된 데이터를 계산하여 하나의 결과를 출력합니다.

인공 뉴런의 구조





각 요소의 역할

① 입력(Input)

입력은 뉴런으로 들어오는 데이터입니다.

예를 들어 학생의 시험 성적이라면

수학 = 90점

영어 = 80점

과학 = 95점

이 각각 하나의 입력이 됩니다.

② 가중치(Weight)

가중치는 각 입력이 얼마나 중요한지를 나타냅니다.

예를 들어 대학 입시에서

수학 : 50%

영어 : 30%

과학 : 20%

처럼 과목마다 반영 비율이 다르다면, 이것이 바로 가중치와 비슷한 개념입니다.

- 중요한 입력 → 큰 가중치
- 덜 중요한 입력 → 작은 가중치

③ 가중합(Weighted Sum)

입력과 가중치를 곱한 후 모두 더하고, 편향을 더합니다.

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

예를 들어

- 수학: 90점 × 0.5
- 영어: 80점 × 0.3
- 과학: 95점 × 0.2

를 모두 더한 값이 가중합입니다.

④ 편향(Bias)

편향은 기준을 조정하는 값입니다.

예를 들어

모든 학생에게 기본점수 5점을 추가한다.
와 같은 역할을 합니다.

편향 덕분에 뉴런은 더 유연하게 학습할 수 있습니다.

⑤ 활성화 함수(Activation Function)

활성화 함수는 가중합을 새로운 출력값으로 변환 합니다.

예를 들어

가중합 = 2.4

↓

활성화 함수

↓

출력 = 0.92

이렇게 변환된 출력은 다음 뉴런의 입력으로 전달됩니다.

또한 활성화 함수는 신경망이 복잡한 문제를 학습할 수 있도록 비선형성을 추가하는 핵심 역할을 합니다.

⑥ 출력(Output)

최종 계산 결과입니다.

예를 들어

고양이일 확률 = 0.98

또는

스팸메일일 확률 = 0.99

처럼 출력됩니다.

인공 뉴런의 동작 과정

입력 데이터

|

▼

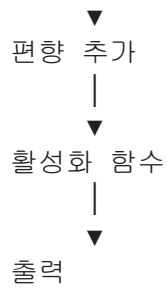
가중치 적용

|

▼

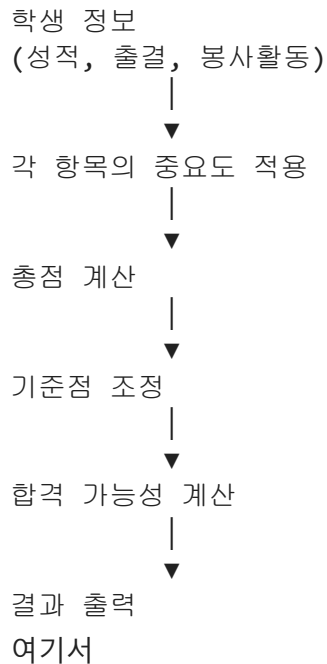
가중합 계산

|



인공 뉴런을 설명하기 좋은 비유

인공 뉴런을 입학사정관에 비유할 수 있습니다.



- 학생 정보 → 입력(Input)
- 과목 반영 비율 → 가중치(Weight)
- 총점 → 가중합
- 기본 가산점 → 편향(Bias)
- 합격 가능성 계산 → 활성화 함수
- 합격 여부 → 출력(Output)

에 해당합니다.

핵심 정리

인공 뉴런은 입력 데이터를 받아 가중치를 적용하고, 편향을 더한 뒤, 활성화 함수를 통해 출력값을 만드는 신경망의 가장 작은 계산 단위입니다.

학생들에게는 다음 문장을 기억하게 하면 좋습니다.

"입력을 받아 계산하고, 판단하여 결과를 내는 작은 계산기 하나가 바로 인공 뉴런이다."

이 개념을 이해하면, 여러 개의 인공 뉴런이 연결된 **신경망(입력층 → 은닉층 → 출력층)**의 구조도 자연스럽게 이해할 수 있습니다.

In []:

03. 머신러닝이란?

중·고등학생에게 "**학습(Learning)**"을 설명할 때는 어려운 수학보다 **사람이 공부하는 과정**과 연결하면 이해가 쉽습니다.

학습이란 무엇인가?

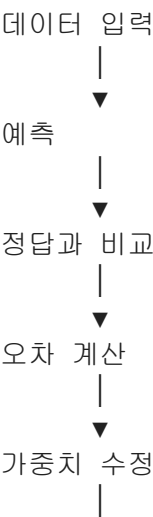
학습이란 컴퓨터가 데이터를 보면서 스스로 규칙을 찾아 예측을 점점 더 정확하게 만드는 과정입니다.

사람이 공부를 통해 실력이 향상되듯이, 신경망도 틀린 부분을 조금씩 수정하면서 점점 더 똑똑해집니다.

사람의 학습과 신경망의 학습 비교

사람	신경망
문제를 푼다.	데이터를 입력한다.
답을 확인한다.	예측값을 계산한다.
틀린 이유를 찾는다.	정답과 비교하여 오차를 계산한다.
다시 공부한다.	가중치를 수정한다.
실력이 향상된다.	예측 정확도가 향상된다.

신경망은 어떻게 학습할까?



▼
다시 예측

이 과정을 수천 번, 수만 번 반복하면서 점점 더 정확한 결과를 얻습니다.

예시 : 강아지 사진 구분하기

처음에는 강아지 사진을 보고

강아지 사진

↓

예측 : 고양이 (❌)
처럼 틀릴 수 있습니다.

정답은

정답 : 강아지
입니다.

그러면 신경망은

■ "왜 틀렸을까?"

를 계산하여 **가중치(Weight)** 를 조금 수정합니다.

다음에는

강아지 사진

↓

예측 : 강아지 (○)
처럼 더 정확한 결과를 냅니다.

시험공부 비유

학생이 수학 문제를 푼다고 생각해 보겠습니다.

문제를 푼다.

↓

틀렸다.

↓

오답노트를 만든다.

↓

다시 공부한다.

↓

다음 시험에서는 맞힌다.
신경망도 거의 같은 과정을 거칩니다.

데이터를 본다.

↓

예측한다.

↓

오차를 계산한다.

↓

가중치를 수정한다.

↓

다시 예측한다.

학습 과정에서 바뀌는 것은?

많은 학생들이 "학습하면서 무엇이 바뀌나요?"라는 질문을 합니다.

정답은 **가중치(Weight)**와 **편향(Bias)** 입니다.

처음

가중치 = 랜덤

↓

학습

↓

가중치 수정

↓

더 정확한 예측
즉,

- 데이터는 바뀌지 않습니다.
- 신경망의 구조도 바뀌지 않습니다.
- 가중치와 편향이 조금씩 바뀌면서 성능이 향상됩니다.

핵심 정리

신경망의 학습이란, 데이터를 이용해 예측을 하고, 정답과의 오차를 계산한 뒤, 가중치와 편향을 조금씩 수정하는 과정을 반복하여 점점 더 정확한 모델을 만드는 것입니다.

"신경망의 학습은 사람이 틀린 문제를 고쳐 가며 공부하는 과정과 같다. 예측 → 오차 확인 → 수정 → 다시 예측을 반복하면서 점점 더 똑똑해진다."

In []:

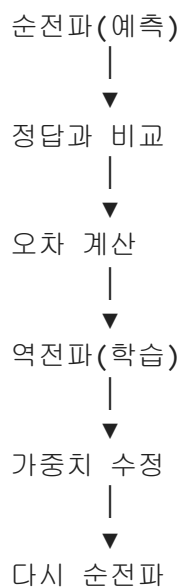
04. 학습과정: 순전파와 역전파의 학습 사이클

"순전파는 지금까지 배운 내용으로 답을 예측하는 과정이고, 역전파는 틀린 이유를 찾아 가중치를 수정하며 배우는 과정이다."

순전파와 역전파의 관계

순전파는 '예측'을 하는 과정이고, 역전파는 '학습'을 하는 과정입니다.

즉, 두 과정은 항상 짝을 이루어 반복됩니다.



1. 순전파(Forward Propagation)

순전파에서는 현재의 가중치와 편향을 이용하여 **예측값을 계산**합니다.

예를 들어,

입력 : 고양이 사진

↓

신경망

↓

예측 : 고양이일 확률 0.82
여기까지가 순전파입니다.

2. 오차 계산

예측이 끝나면 정답과 비교합니다.

예측 : 0.82

정답 : 1

↓

오차 = 0.18

오차가 크면 신경망은 아직 제대로 학습하지 못한 것입니다.

3. 역전파(Backpropagation)

역전파는

"왜 틀렸는지 계산하여 가중치와 편향을 수정하는 과정"입니다.

즉,

오차

↓

출력층 수정

↓

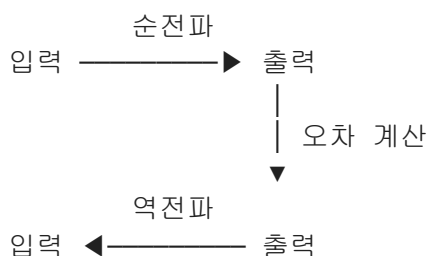
은닉층 수정

↓

입력층 방향으로 전달

하면서 가중치를 조금씩 변경합니다.

순전파와 역전파를 그림으로 보면



- **순전파:** 입력 → 출력
- **역전파:** 출력 → 입력

정보가 흐르는 방향이 서로 반대입니다.

설명하기 좋은 비유

시험공부를 예로 들어 보겠습니다.

① 순전파

시험 문제를 푼다.

↓

답을 제출한다.

② 오차 계산

정답을 확인한다.

↓

틀린 문제를 찾는다.

③ 역전파

왜 틀렸는지 공부한다.

↓

공부 방법을 수정한다.

④ 다시 순전파

다음 시험을 본다.

↓

더 높은 점수를 받는다.

학습 과정 전체

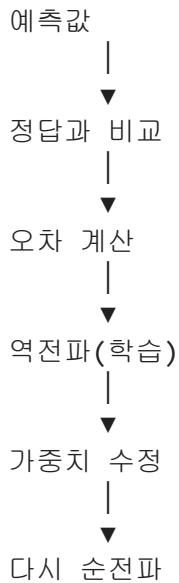
입력 데이터

↓

순전파(예측)

↓

▼



이 과정을 수천 번 반복하면 신경망의 예측이 점점 정확해집니다.

핵심 정리

순전파	역전파
입력에서 출력 방향으로 계산	출력에서 입력 방향으로 오차 전달
예측값 생성	가중치와 편향 수정
현재 모델의 성능 확인	모델을 더 좋게 학습

학생들에게 기억하게 할 한 문장

순전파는 "답을 예측하는 과정"이고, 역전파는 "틀린 이유를 이용해 배우는 과정"입니다. 이 두 과정이 반복되면서 신경망은 점점 더 정확한 예측을 할 수 있게 됩니다.

In []:

04. 순전파 과정

중·고등학생에게는 순전파(Forward Propagation)를 "현재 알고 있는 내용으로 답을 예측하는 과정"이라고 설명하면 가장 이해하기 쉽습니다.

순전파(Forward Propagation)란?

순전파는 입력 데이터를 이용하여 현재의 가중치와 편향으로 예측값을 계산하는 과정입니다.

쉽게 말하면,

"지금까지 학습한 내용을 이용하여 문제의 답을 맞춰 보는 과정"입니다.

순전파의 흐름

입력 데이터



가중치 적용



가중합 계산
($z = XW + b$)



활성화 함수



예측값 출력

정보가 **입력층** → **은닉층** → **출력층**으로 이동하기 때문에 **순전파(Forward Propagation)** 라고 부릅니다.

예를 들어

고양이 사진을 입력했다고 가정해 보겠습니다.

고양이 사진



신경망



고양이일 확률 = 0.96
또는

손글씨 숫자 이미지



신경망



숫자 7일 확률 = 0.99

이처럼 **예측 결과를 계산하는 과정이 순전파**입니다.

순전파에서 하는 일

신경망의 한 뉴런에서는 다음과 같은 계산이 이루어집니다.

1. 입력을 받는다.
2. 입력에 가중치를 곱한다.

3. 모두 더하고 편향을 더한다.
4. 활성화 함수를 적용한다.
5. 다음 뉴런으로 전달한다.

이를 모든 뉴런이 동시에 수행합니다.

XOR 예제에서의 순전파

입력이

$X = [1, 0]$

이라면

순전파는 다음과 같이 진행됩니다.

입력

$[1, 0]$



은닉층 계산



출력층 계산



예측 = 0.87

정답이 1이라면,

현재 신경망은 **1일 가능성이 높다고** 예측한 것입니다.

설명하기 좋은 비유

시험을 보는 학생을 생각해 봅시다.

문제를 읽는다.



지금까지 공부한 내용을 떠올린다.



답을 적는다.

이것이 사람의 **예측 과정**입니다.

신경망도 마찬가지입니다.

데이터를 입력한다.



현재 가중치로 계산한다.



▼
예측 결과를 출력한다.
여기까지는 **정답과 비교하지 않습니다.**

순전파와 역전파의 차이

순전파	역전파
예측값을 계산한다.	오차를 이용해 가중치를 수정한다.
입력 → 출력 방향	출력 → 입력 방향
답을 맞춰 보는 과정	틀린 이유를 분석하고 배우는 과정

핵심 정리

순전파(Forward Propagation)는 입력 데이터를 현재의 가중치와 편향으로 계산하여 예측값을 만드는 과정입니다.

학생들에게는 다음 한 문장을 기억하게 하면 좋습니다.

"순전파는 지금까지 배운 내용으로 답을 예측하는 과정이고, 역전파는 틀린 이유를 찾아 가중치를 수정하며 배우는 과정이다."

In []:

05. 역전파 과정

역전파(Backpropagation)란?

역전파는 예측 결과와 정답의 차이(오차)를 이용하여 가중치와 편향을 수정하는 학습 과정입니다.

쉽게 말하면,

"틀린 이유를 찾아 다음에는 더 잘 맞히도록 배우는 과정" 입니다.

역전파는 왜 필요할까?

신경망이 처음에는 가중치를 무작위(Random)로 가지고 있습니다.

예를 들어,

고양이 사진

↓

예측 : 개 (❌)
처럼 틀릴 수 있습니다.

그렇다면 신경망은

"왜 틀렸을까?"

를 계산하여 가중치를 조금 수정합니다.

이 과정이 **역전파**입니다.

역전파의 과정

입력 데이터



순전파 (예측)



예측값



정답과 비교



오차 계산



역전파



가중치 수정

역전파는 오차를 이용하여 가중치를 수정하는 과정 입니다.

왜 '역(Back)' 전파일까?

순전파에서는 정보가

입력층



은닉층



출력층

으로 이동합니다.

반면 역전파에서는

입력층





처럼 출력층에서 입력층 방향으로 오차를 전달 합니다.

그래서 이름이 **Backpropagation**(역전파) 입니다.

설명하기 좋은 비유

시험을 본 학생을 생각해 봅시다.

① 시험을 본다.

문제를 푼다.

↓

② 채점한다.

90점

↓

③ 틀린 문제를 분석한다.

왜 틀렸지?

↓

④ 다시 공부한다.

다음 시험은 95점

↓

⑤ 계속 반복한다.

실력이 점점 향상됩니다.

신경망도 완전히 같은 원리입니다.

XOR 예제로 보면

예를 들어

입력 : (1,0)

정답 : 1

예측 : 0.35

이라면

오차는

$$1 - 0.35 = 0.65$$

입니다.

역전파는

이 **0.65**라는 오차를 이용하여

출력층 가중치 수정

↓

은닉층 가중치 수정

을 수행합니다.

다음에는

예측 : **0.60**

다시 학습하면

예측 : **0.83**

또 학습하면

예측 : **0.97**

처럼 정답에 가까워집니다.

순전파와 역전파의 관계

입력 데이터



순전파

(예측)



예측값



정답과 비교



오차 계산



역전파

(가중치 수정)



다시 순전파

이 과정을 수천 번 반복하면 신경망은 점점 더 정확한 예측을 하게 됩니다.

핵심 정리

역전파는 예측 결과와 정답의 차이를 이용하여 가중치와 편향을 수정하는 학습 과정입니다. 오차를 출력층에서 입력층 방향으로 전달하면서 각 뉴런의 가중치를 조금씩 수정하기 때문에 '역전파'라고 부릅니다.

"순전파는 답을 예측하는 과정이고, 역전파는 틀린 이유를 분석하여 다음에는 더 잘 맞히도록 배우는 과정이다."

In []:

06. Epoch와 반복 학습

1. Epoch(에포크)란?

에포크(Epoch)는 학습 데이터 전체를 한 번 모두 사용하여 학습하는 것을 의미합니다.

예를 들어,

- 학습 데이터 : 1,000장
- 에포크 : 1

이라면,

1,000장의 데이터를 모두 한 번씩 이용하여 모델을 학습했다는 뜻입니다.

2. 반복 학습(Iteration)이란?

Iteration(반복)은 모델의 가중치를 한 번 업데이트하는 과정입니다.

대부분의 딥러닝에서는 데이터를 한 번에 모두 사용하지 않고 **미니배치(Mini Batch)** 단위로 나누어 학습합니다.

예를 들어

- 데이터 : 1,000개
- 배치 크기(Batch Size) : 100

이라면

1회 반복에서

→ 데이터 100개를 이용하여

- 순전파(예측)
- 오차 계산

- 역전파
- 가중치 수정

을 수행합니다.

이것이 **Iteration 1회**입니다.

3. Epoch와 Iteration의 관계

예를 들어

학습 데이터 : 1,000개

배치 크기 : 100

이라면

$$1000 \div 100 = 10$$

즉

1 Epoch = 10 Iteration

입니다.

그림으로 나타내면

Epoch 1

Iteration 1
데이터 1~100

↓

Iteration 2
데이터 101~200

↓

Iteration 3
데이터 201~300

↓

...

↓

Iteration 10
데이터 901~1000

1 Epoch 완료

4. 여러 Epoch를 학습하면?

만약

Epoch = 5

라면

Epoch 1
데이터 전체 학습

↓

Epoch 2
다시 처음부터 데이터 전체 학습

↓

Epoch 3

↓

Epoch 4

↓

Epoch 5

즉 같은 데이터를 여러 번 반복하여 학습합니다.

왜냐하면 한 번만 보면 충분히 패턴을 익히지 못하기 때문입니다.

5. 왜 여러 번 반복할까?

예를 들어 학생이 영어 단어를 외운다고 생각해 봅시다.

1회 읽음

apple
banana
orange
...

거의 기억하지 못합니다.

5번 읽으면

apple
banana

orange

...

점점 기억합니다.

20번 읽으면

거의 외우게 됩니다.

AI도 마찬가지입니다.

한 번 데이터를 보는 것으로는 충분하지 않기 때문에 여러 Epoch를 수행합니다.

6. 너무 많이 학습하면?

에포크를 계속 증가시키면

초기에는

오차 ↓

정확도 ↑

하지만 너무 많이 학습하면

훈련 데이터는 거의 외움

↓

새로운 데이터에서는 성능이 떨어짐

이를 **과적합(Overfitting)**이라고 합니다.

그래서 적절한 Epoch에서 학습을 멈추는 것이 중요합니다.

7. 전체 흐름

데이터 준비

↓

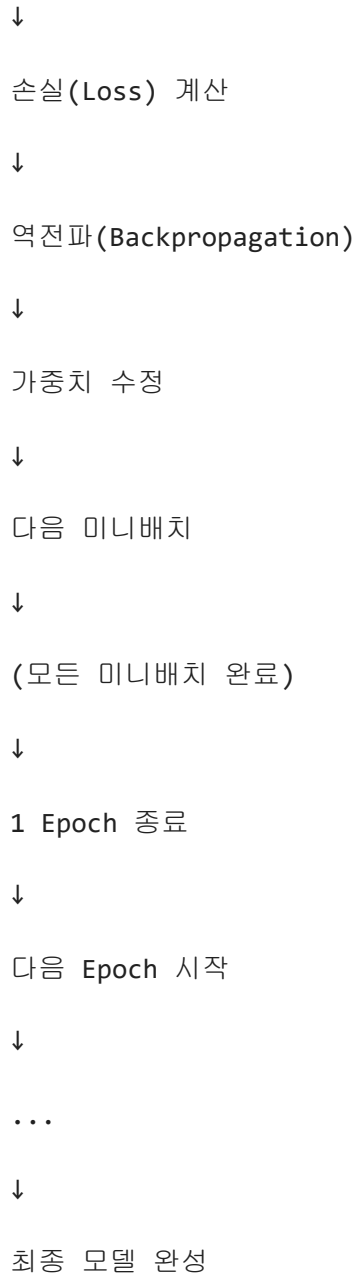
Epoch 시작

↓

미니배치 선택

↓

순전파(예측)



핵심 정리

용어	의미	예시
데이터셋(Dataset)	전체 학습 데이터	1,000개 이미지
배치(Batch)	한 번에 사용하는 데이터 묶음	100개
반복(Iteration)	배치 1개를 사용해 순전파→역전파→가중치 업데이트를 한 번 수행	데이터 100개 학습
에포크(Epoch)	전체 데이터셋을 한 번 모두 학습	데이터 1,000개 전체 학습
관계	Iteration 수 = 데이터 수 ÷ 배치 크기	$1,000 \div 100 = 10$ Iteration = 1 Epoch

다음과 같이 비유하면 이해하기 쉽습니다.

- 데이터셋: 한 권의 문제집
- 배치: 한 번에 푸는 10쪽
- Iteration: 10쪽을 풀고 오답을 확인하며 공부 방법을 조금 수정하는 과정
- Epoch: 문제집을 처음부터 끝까지 한 번 모두 푸는 것

이 비유를 사용하면 "한 번의 에포크는 여러 번의 반복(Iteration)으로 이루어지며, 반복마다 모델이 조금씩 똑똑해진다"는 점을 학생들이 직관적으로 이해할 수 있습니다.

07. XOR 문제로 보는 은닉층의 필요성

1. XOR(배타적 OR)이란?

XOR은 두 입력이 서로 다를 때만 1을 출력하는 논리 연산입니다.

x ₁	x ₂	XOR 출력
0	0	0
0	1	1
1	0	1
1	1	0

즉,

- 둘이 같으면 → 0
- 둘이 다르면 → 1

2. 좌표로 표현해 보기

입력을 좌표라고 생각하면 다음과 같습니다.

	x ₂ ↑	
1	○(0,1)=1	●(1,1)=0
0	●(0,0)=0	○(1,0)=1
	0-----1 → x ₁	

- ○ : 출력 1
- ● : 출력 0

3. 직선 하나로는 분리할 수 없다

예를 들어 직선 하나를 그어

○와 ●를 나누려 한다.

하지만



처럼 대각선 방향으로 섞여 있기 때문에

직선 하나로써는 절대 분리할 수 없습니다.

이러한 문제를

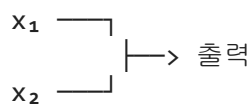
선형 분리(Linearly Separable)가 불가능한 문제

라고 합니다.

4. 단일 뉴런의 한계

은닉층이 없는 신경망은

입력



수식으로는

$$y = f(w_1x_1 + w_2x_2 + b)$$

입니다.

여기서

$$w_1x_1 + w_2x_2 + b = 0$$

은 직선을 의미합니다.

즉,

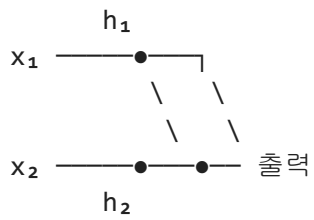
은닉층이 없는 신경망은

직선 하나밖에 만들 수 없습니다.

그래서 XOR은 해결하지 못합니다.

5. 은닉층이 있으면?

은닉층을 하나 추가하면



은닉층의 각 뉴런이

각각 하나의 직선을 만듭니다.

예를 들면

- 뉴런 1 → 직선 A
- 뉴런 2 → 직선 B

를 만들고,

출력층이

이 둘을 조합합니다.

즉

직선 A

+

직선 B

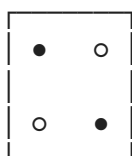
=

복잡한 경계

가 만들어집니다.

6. XOR을 분리하는 모습

은닉층 두 개가 있으면



를

예를 들어

\

/

두 개의 경계로 나누고,

출력층이 이를 조합하여

○

○

와

●

●

를 정확하게 구분합니다.

즉

직선 하나는 안 되지만

직선 여러 개를 조합하면 가능합니다.

7. 왜 '은닉층'이라고 부를까?

입력도 아니고 출력도 아니며,

중간에서 특징을 만들어내기 때문입니다.

예를 들어

입력

↓

"왼쪽인가?"

↓

"위쪽인가?"

↓

"대각선인가?"

↓

출력

처럼

중간에서 새로운 특징을 만들어 냅니다.

이를 **특징 추출(Feature Extraction)**이라고 합니다.

8. 비유로 이해하기

직선 하나로 종이를 자른다고 생각해 봅시다.

직선 한 번



으로는

○ ● ○ ●

를 구분할 수 없습니다.

하지만

두 번 자르면



|

여러 영역으로 나눌 수 있습니다.

은닉층은

 **종이를 여러 번 자를 수 있게 해 주는 역할**

이라고 생각하면 됩니다.

9. 딥러닝에서는?

은닉층이 하나이면

직선 여러 개

를 조합할 수 있습니다.

은닉층이 많아질수록

직선



곡선



복잡한 곡면



매우 복잡한 결정 경계

를 만들 수 있습니다.

그래서

- 필기체 인식
- 얼굴 인식
- 음성 인식
- 생성형 AI

처럼 매우 복잡한 문제도 해결할 수 있습니다.

핵심 요약

- 단일 뉴런(은닉층 없음)은 직선 하나의 결정 경계만 만들 수 있어 XOR 문제를 해결하지 못합니다.
- 은닉층은 여러 뉴런이 각각 다른 직선(또는 경계)을 학습하고, 이를 조합하여 복잡한 결정 경계를 만듭니다.
- XOR 문제는 은닉층이 필요한 이유를 보여 주는 대표적인 예이며, 딥러닝이 복잡한 패턴을 학습할 수 있는 원리를 직관적으로 설명해 줍니다.

중·고등학생 수업에서는 "직선 하나로는 안 되지만, 직선 여러 개를 조합하면 된다."라는 메시지를 먼저 전달한 뒤, 이것이 바로 은닉층의 역할이라고 설명하면 이해도가 크게 높아집니다.

In []:

08. XOR 문제: 신경망으로 XOR 학습하기

입력은 두 개, 은닉층 뉴런은 두 개, 출력은 하나인 가장 작은 신경망입니다.

[실습] 예제 : 2-2-1 신경망으로 XOR 학습하기

입력은 두 개, 은닉층 뉴런은 두 개, 출력은 하나인 가장 작은 신경망

[입력 → 가중치 → 은닉층 → 출력 → 오차 계산 → 역전파 → 가중치 수정 → 다시 입력] 이 과정이 수천 번 반복되면서

[랜덤 가중치 → 조금 수정 → 조금 더 수정 → 오차 감소 → 좋은 가중치]가 되는 것이 학습(Learning)

In [8]: `import numpy as np`

```
# XOR 데이터: 입력: x, 출력: y
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

y = np.array([
    [0],
    [1],
    [1],
    [0]
])

# 시그모이드 함수
def sigmoid(x):
    return 1/(1+np.exp(-x))

# 미분
def sigmoid_derivative(x):
    return x * (1 - x)

# 가중치 초기화: 은닉층 2개, 출력층 1개
#  $y = wx + b$ 에서  $w$ 은 가중치(Weight),  $b$ 은 편향(Bias)
np.random.seed(1)

w1 = np.random.randn(2, 2) #초기화: 은닉층 2개
b1 = np.zeros((1, 2)) #1행 2열의 모든 원소가 0인 NumPy 배열

w2 = np.random.randn(2, 1) #초기화: 출력층 1개
b2 = np.zeros((1, 1))

learning_rate = 0.5 #학습률(Learning Rate): 가중치를 한 번에 얼마나 수정할 것인지

print("\n오차 제공 확인(수정 횟수 : 오차 제공)")
# 학습
for epoch in range(10000): # x->y 4개의 데이터를 이용하여, [순전파 → 역전파 → 가중치 수정] 반복
    # ----- 순전파 -----
    hidden = sigmoid(np.dot(X, w1) + b1) #은닉층의 순전파 (행렬 곱으로 입력과 가중치 곱하기)
    output = sigmoid(np.dot(hidden, w2) + b2) #출력층의 순전파

    # ----- 오차 계산 -----
    error = y - output

    # ----- 역전파 -----
    d_output = error * sigmoid_derivative(output) #출력층 수정량 계산: 시그모이드 함수의 미분
    error_hidden = np.dot(d_output, w2.T) #역전파(Back Propagation): 출력층의 오차 역전파
    d_hidden = error_hidden * sigmoid_derivative(hidden) #은닉층의 수정량 계산

    # ----- 가중치 수정 -----
    w2 += learning_rate * np.dot(hidden.T, d_output) #T는 Transpose(전치, 전치행렬)
    b2 += learning_rate * np.sum(d_output, axis=0)
    w1 += learning_rate * np.dot(X.T, d_hidden)
    b1 += learning_rate * np.sum(d_hidden, axis=0)
```



```
if (epoch % 1000) == 0: #1000번 마다 오차 제곱을 확인
    loss=np.mean(error ** 2)
    print(epoch, ': ', loss)

print("\n예측 결과")
print(output) #10000번 수정된 최종 출력층 값 확인
```

오차 제곱 확인 (수정 횟수 : 오차 제곱)

```
0 : 0.256657767612402
1000 : 0.08141842346858916
2000 : 0.0034498748643492626
3000 : 0.0015639952302803056
4000 : 0.0009946603172000007
5000 : 0.0007245986657864952
6000 : 0.0005680257893845742
7000 : 0.00046619518098476345
8000 : 0.0003948276574462906
9000 : 0.0003421098233023137
```

예측 결과

```
[[0.01546415]
 [0.98366378]
 [0.98334025]
 [0.02056836]]
```

In []: